

Family Hub

Documentacion tecnica del bloque Obsidian

Namespace: Rock.Blocks.FamilyHub

Rock: 18.1 — rama `hotfix-18.1`

Fecha: 2026-04-18

Archivos: FamilyHub.cs (backend) · FamilyHub.obs (frontend)

Family Hub — Documentación técnica

Bloque Obsidian que permite al usuario autenticado ver y administrar los miembros de su familia y sus Known Relationships (parentescos) desde el portal de VidaReal.

Ubicación de archivos:

- Backend: `Rock.Blocks/FamilyHub/FamilyHub.cs`
- Frontend: `Rock.JavaScript.Obsidian.Blocks/src/FamilyHub/FamilyHub.obs`
- Namespace C#: `Rock.Blocks.FamilyHub`
- Categoría Rock: `Custom`

1. Visión general

Family Hub expone una UI estilo "tarjetas" donde el usuario puede:

1. **Ver** a todos los miembros de su grupo familiar primario + personas relacionadas por Known Relationship.
2. **Agregar** un nuevo miembro (crea un `Person` + lo inserta en la familia o como KR-only).
3. **Editar** los datos personales (nombre, fecha de nacimiento, género, email, teléfono móvil, foto, parentesco).
4. **Gestionar parentesco** usando dos sistemas: - **Known Relationship** (rol configurable: Tío, Primo, Amigo, etc.). - **Estado civil** (marital status) que aplica `MaritalStatusValueId` en ambos perfiles y fuerza pertenencia a la familia como Adulto.

La eliminación de miembros **no** se hace desde el bloque — se deriva por email a `soporteapp@vidareal.tv` (regla de negocio).

2. Configuración del bloque (BlockAttributes)

Attribute Key	Tipo	Requerido	Descripción
<code>AvailableKnownRelationshipRoles</code>	<code>CustomEnhancedListField</code>	No	Lista de roles de Known Relationship disponibles. Si queda vacío, se permiten todos excepto Owner. Pobra el selector "Parentesco conmigo".
<code>AvailableMaritalStatusOptions</code>	<code>CustomEnhancedListField</code>	No	Lista de estados civiles (<code>DefinedValue</code> guids del tipo <code>b4b92c3f-a935-40e1-a00b-ba484ead613b</code>). Si queda vacío, no se muestra sección marital.

Los `ListSource` SQL están embebidos como constantes en `FamilyHub.ListSource` para poblar el picker del admin.

3. Arquitectura — backend (`FamilyHub.cs`)

3.1 Clase base

```
public class FamilyHub : RockBlockType
```

3.2 Inicialización — `GetObsidianBlockInitialization()`

Flujo:

1. Si no hay usuario autenticado → devuelve `InitBag { notLogged = true, ... } .`
2. Obtiene el `GroupId` primario de familia del usuario (`GetPrimaryFamilyGroupId`).
3. Resuelve los roles de Known Relationship disponibles según configuración (`GetKnownRelationshipRoles`).
4. Si no hay familia primaria → devuelve `InitBag` con `statusHtml` de advertencia.
5. Caso normal → enumera miembros con `GetPeopleMerged(...)` y devuelve el `InitBag` completo.

3.3 Block Actions

Action	Entrada	Salida	Responsabilidad
GetEdit	GetEditRequestBag { personId }	GetEditResponseBag { model }	Carga el <code>EditModelBag</code> para pre-poblar el modal de edición (datos personales + teléfono separado en código y número + parentesco actual).
SaveMember	SaveMemberRequestBag	SaveMemberResponseBag { statusHtml, members[] }	Crea o actualiza <code>Person</code> + aplica foto + aplica marital o Known Relationship + devuelve lista actualizada.

3.4 DTOs (Bags)

- `InitBag` — payload de inicialización.
- `MemberListItemBag` — fila de la grilla de miembros (datos ya formateados para UI).
- `EditModelBag` — modelo del modal de edición.
- `GetEditRequestBag` / `GetEditResponseBag` — wrappers de `GetEdit`.
- `SaveMemberRequestBag` / `SaveMemberResponseBag` — wrappers de `SaveMember`.
- `PersonRow` (privada) — proyección LINQ intermedia para `GetPeopleMerged`.

Nota de estilo: las propiedades están en `camelCase` (no `PascalCase`) porque el front consume directo sin transformación. No es idiomático C# pero es consistente con el resto del bloque.

3.5 Helpers clave

Helper	Propósito
<code>GetFamilyGroupId</code>	Devuelve <code>GroupTypeCache.GetFamilyGroupType()?.Id</code> . Único punto de acceso al Id del GroupType "Family"; devuelve <code>int?</code> para que los callers manejen null explícitamente.
<code>GetPrimaryFamilyGroupId</code>	Devuelve el <code>GroupId</code> de la familia activa primaria de una persona.
<code>IsPersonInFamily</code>	Chequea si una persona es miembro activo del grupo familiar dado.
<code>CanEditPerson</code>	Autorización: permite editar si el target está en la familia o en el KR-group del owner.
<code>AddPersonToFamily</code>	Inserta/actualiza la membresía en la familia, eligiendo rol Adult/Child según edad o flag <code>preferChildRole</code> . Limpia otras membresías familiares del target.
<code>RemovePersonFromFamily</code>	Elimina la membresía y desactiva el grupo si quedó vacío.
<code>EnsurePersonHasPrimaryFamily</code>	Si la persona quedó huérfana (sin familia), le crea una nueva o reactiva la existente. Resuelve el rol Adulto antes de crear el Group para no dejar grupos huérfanos.
<code>RemoveFamilyMembershipsExcept</code>	Limpia membresías familiares duplicadas borrándolas (persona solo puede estar en una familia primaria).
<code>DeactivateFamilyGroupIfNoActiveMembers</code>	Tras borrar miembros, marca el grupo como inactivo si ya no tiene miembros activos.
<code>GetMyKnownGroupId</code> / <code>GetKnownGroupIdByOwnerPerson</code> / <code>EnsureKnownGroupForOwnerPerson</code>	Resolución y creación del grupo de Known Relationship del owner.
<code>NormalizeKnownRelationshipRoleId</code>	Valida que un RoleId pertenezca al GroupType correcto y esté dentro de los roles permitidos por configuración.
<code>GetKnownChildRoleId</code> / <code>IsKnownRelationshipChildRole</code>	Detecta el rol "Child" de KR (por guid o por nombre ES/EN).
<code>SaveKnownRelationshipFromMeToPerson</code>	Reescribe los KR del owner → target: borra los previos y crea el nuevo (si <code>relationshipRoleId = null</code> , solo borra).
<code>GetMaritalRelationshipValue</code>	Si el target es Adulto en mi familia y comparte <code>MaritalStatusValueId</code> configurado → devuelve <code>"marital:<guid>"</code> .

Helper	Propósito
<code>ApplyMaritalStatus</code>	Aplica <code>MaritalStatusValueId</code> a ambos (owner + target) y agrega target a la familia como Adulto.
<code>GetMobilePhoneParts</code> / <code>SaveMobilePhoneV2</code>	Lectura y escritura del teléfono móvil (separado en <code>CountryCode</code> + <code>Number</code>).
<code>ResolvePhotoBinaryFile</code>	Resuelve <code>BinaryFile</code> a partir de GUID o ID numérico.
<code>ApplyPhotoFromBag</code>	Única entrada para persistir foto (marca no-temporal, fuerza tipo <code>PERSON_IMAGE</code> , optimiza imagen, asigna <code>PhotoId</code>).
<code>OptimizeProfileImageBinaryFile</code> / <code>ResizeToMaxDimension</code> / <code>ApplyExifOrientation</code> / <code>SaveAsJpeg</code>	Pipeline de optimización: corrige orientación EXIF, redimensiona a 1200px máx, guarda JPEG calidad 82.
<code>BuildPhotoUrl</code>	Construye <code>/GetImage.ashx?id=...&w=X&h=X&mode=crop</code> .
<code>GetInitials</code>	Extrae iniciales (primera letra del primer nombre + primera del último apellido).

3.6 Guids constantes

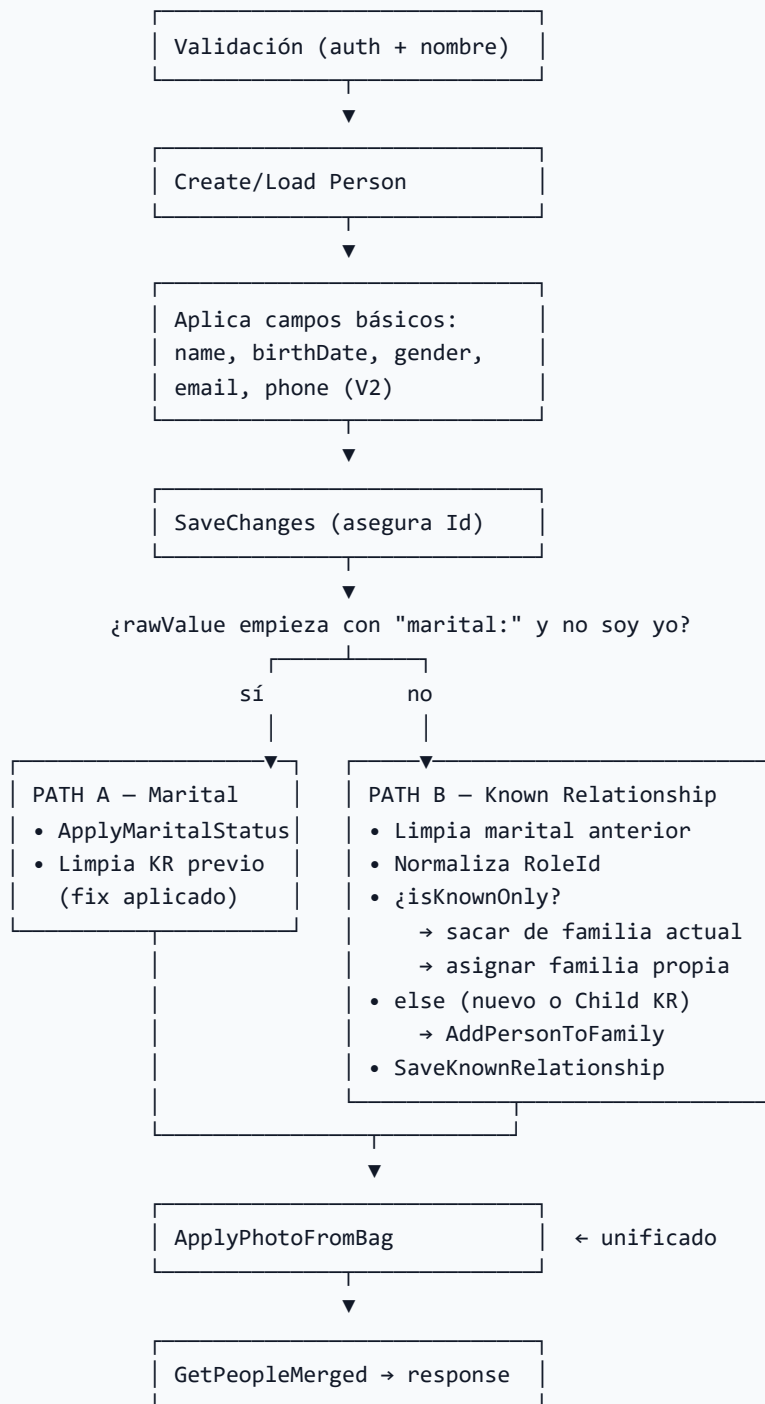
```

KnownRelationshipGroupTypeGuid = "E0C5A0E2-B7B3-4EF4-820D-BBF7F9A374EF"
KnownRelationshipOwnerRoleGuid = SystemGuid.GroupRole.GROUPROLE_KNOWN_RELATIONSHIPS_OWNER
KnownRelationshipChildRoleGuid = SystemGuid.GroupRole.GROUPROLE_KNOWN_RELATIONSHIPS_CHILD
FamilyAdultRoleGuid           = SystemGuid.GroupRole.GROUPROLE_FAMILY_MEMBER_ADULT
FamilyChildRoleGuid           = SystemGuid.GroupRole.GROUPROLE_FAMILY_MEMBER_CHILD
MaritalPrefix                  = "marital:"

```

El prefijo `"marital:"` se concatena al `Guid` del `DefinedValue` de estado civil para distinguir valores marital vs. RoleId numéricos en un único campo `relationshipValue`.

4. Flujo de **SaveMember**



5. Arquitectura — frontend (`FamilyHub.obs`)

5.1 Composición

- `Panel` (sin padding) envolviendo el layout `VidaReal`.
- Grid de tarjetas (`fhcard`) con foto, nombre, edad, chips y datos de contacto.
- Modal de edición (`fhmodal`) con columnas: formulario izquierdo + foto sticky a la derecha.

5.2 Estado (refs)

- `members` , `statusHtml` , `relationshipRoles` , `genderOptions` — estado inicial del `InitBag` .
- `modalOpen` , `busy` , `editError` , `photoUploadBusy` , `saveAttempted` — flags de UI.
- `editModel` — `EditModel` plano que respalda al formulario.
- `birthDay` , `birthMonth` , `birthYear` + `*Touched` — entrada separada de 3 inputs (DD / MM / AAAA).
- `photoFile` (`ListItemBag` de `ImageEditor`).
- `phoneMenuOpen` — popover del selector de código país.

5.3 Validación

- **Campos:** nombre, apellido, sexo, email, teléfono, parentesco (si no soy yo). Se validan sólo tras primer `save()` (via `saveAttempted`).
- **Fecha de nacimiento:** validada por inputs 3-parts con mensajes:
- `birthDayError` — 1–31.
- `birthMonthError` — 1–12.
- `birthYearError` — 1900 .. año actual.
- `birthDateValidText` — "DD de MES de AAAA" cuando la fecha es válida.

5.4 Pipeline de foto

1. `triggerPhotoPicker()` → dispara el `<input type="file">` del `ImageEditor` oculto.
2. El usuario recorta en el control nativo (`ImageEditor`).
3. `@cropped` → `beginPhotoUploadState()` — bloquea la UI, arranca timeout 25s.
4. `@update:modelValue` → `onPhotoModelValueUpdated()` — limpia el timeout y muestra preview.
5. El valor del `photoFile` puede ser un **GUID** (flujo normal) o un **ID numérico**; `photoPreviewUrl` / `photoBinaryFileGuid` / `photoBinaryFileId` desambiguan.

5.5 Teléfono

- Código país separado (selector con `+502 / +503 / +1` , default `502`).
- Solo se admiten dígitos (`onPhoneNumberInput`).
- Backend guarda en `PhoneNumber.CountryCode` (string) + `PhoneNumber.Number` (limpio vía `PhoneNumber.CleanNumber`).

5.6 Estilos

- Design tokens CSS en `:root` (`--vr-*`).
 - Fuerza modo light en todas las variables Rock (`--color-interface-*`) para evitar tema oscuro heredado.
 - Responsive: grids colapsan a 2/1 columnas a 1024px / 640px.
-

6. Bugs encontrados y correcciones aplicadas

6.1 Bugs corregidos en este commit

#	Severidad	Descripción	Corrección
1	Alta	Al cambiar el parentesco a un estado civil (Path A), los Known Relationships previos (ej. "Tío", "Primo") no se limpiaban — la persona conservaba roles contradictorios.	<code>SaveMember</code> (Path A) ahora llama <code>SaveKnownRelationshipFromMeToPerson(..., null, ...)</code> después de aplicar el marital.
1b	Alta	Al cambiar el parentesco de estado civil a Known Relationship (Path B), el <code>MaritalStatusValueId</code> persistía en ambos adultos. La limpieza dependía de <code>GetMaritalRelationshipValue</code> , que sólo retornaba no-null si el target seguía siendo Adulto activo en la familia actual Y la opción marital seguía habilitada en la config del bloque — en cuanto fallaba una de esas precondiciones, el marital quedaba residual.	Path B ahora limpia <code>MaritalStatusValueId</code> directamente si el valor presente está en <code>GetConfiguredMaritalDefinedValueIds()</code> . El <code>currentPerson</code> se limpia sólo si su marital coincidía con el del target (evita romper otra relación preexistente).
1c	Alta	Al pasar el parentesco de KR a estado civil (Casado / Unido), si la persona venía desde fuera de la familia (KR-only) con <code>BirthDate < 18</code> o con un GroupMember previo como Child, quedaba incorrectamente como Child en la familia del cónyuge. <code>AddPersonToFamily</code> decidía el rol por edad (<code>BirthDate.Age() < 18</code>) sin distinguir el contexto marital.	Se agregó el parámetro <code>forceAdultRole</code> a <code>AddPersonToFamily</code> . <code>ApplyMaritalStatus</code> ahora lo invoca con <code>forceAdultRole: true</code> , saltando la inferencia por edad. Un cónyuge siempre queda como Adulto en la familia.
2	Alta	N+1 queries en <code>GetPeopleMerged</code> : por cada miembro se ejecutaban 3-4 queries adicionales (<code>GetMobilePhone</code> , <code>GetMaritalRelationshipValue</code> , <code>PersonService.Get</code> para el badge, <code>GetConfiguredMaritalStatusOptions</code> dentro del loop). Una familia de 10 miembros generaba ~40 roundtrips.	Batching: se cargan en proyección <code>MaritalStatusValueId</code> + <code>IsAdultInCurrentFamily</code> ; una sola query de <code>PhoneNumber</code> y una de roles KR para todos los personIds. Las opciones marital se cachean fuera del loop. Resultado: ≤ 5 queries totales, independiente del tamaño de la familia.
3	Media	Código duplicado: la lógica de guardar foto (~30 líneas) aparecía 2 veces (una en Path A, otra en Path B). Riesgo: divergencia al modificar.	Extraído a <code>ApplyPhotoFromBag(rockContext, person, bag)</code> + helper <code>ResolvePhotoBinaryFile</code> . Se invoca una sola vez al final de <code>SaveMember</code> .

#	Severidad	Descripción	Corrección
4	Media	NPE potencial en <code>GroupTypeCache.GetFamilyGroupType().Id</code> (7 llamadas sin guard). Si Rock está mal configurado al arranque, lanzaba NRE.	Se creó el helper <code>GetFamilyGroupId()</code> que devuelve <code>int?</code> ; todos los callers retornan temprano si es <code>null</code> .
5	Media	<code>DeactivateFamilyMembershipsExcept</code> no desactivaba — eliminaba las memberships (<code>groupMemberService.Delete(...)</code>). El nombre era engañoso y confundía a quien leyera el código.	Renombrado a <code>RemoveFamilyMembershipsExcept</code> + docstring que describe el comportamiento real (borra, luego desactiva el grupo si quedó vacío).
6	Media	<code>EnsurePersonHasPrimaryFamily</code> creaba y guardaba el <code>Group</code> antes de resolver el rol Adulto. Si el rol no existía, quedaba un grupo huérfano en BD.	Ahora se resuelve <code>adultRole</code> antes del <code>Add</code> . El <code>Group</code> + <code>GroupMember</code> se crean en memoria y se persisten juntos en un único <code>SaveChanges()</code> — atómico.
7	Baja	Propiedad <code>mobile</code> duplicada en <code>EditModelBag</code> y <code>SaveMemberRequestBag</code> marcada como "compat" pero ya no se usaba (el front usa <code>phoneCountryCode</code> / <code>phoneNumber</code>).	Eliminada en C# + en el frontend se removió el fallback <code>parsePhone(m.mobile)</code> y la función <code>parsePhone</code> quedó borrada.
8	Baja	Helpers muertos tras el refactor: <code>GetKnownRelationFromMe</code> (reemplazado por <code>batch</code>) y <code>GetMobilePhone</code> (reemplazado por <code>diccionario</code>).	Eliminados. Queda <code>GetMobilePhoneParts</code> (sigue usándose en <code>GetEdit</code>).

6.2 Issues conocidos (NO corregidos — requieren decisión de producto)

#	Severidad	Descripción	Recomendación
1	Baja	<code>OptimizeProfileImageBinaryFile</code> captura cualquier <code>Exception</code> y solo logea. Si la conversión falla, el archivo queda en su formato original (puede ser PNG grande sin optimizar).	Considerar fallback a <code>resize</code> sin <code>reencode</code> , o <code>fail-fast</code> con mensaje al usuario.
2	Baja	<code>GetInitials</code> (C#) y <code>photoPreviewInitials</code> (TS) duplican la misma lógica de extracción. Coinciden hoy, pero pueden divergir.	Documentar o centralizar en el backend.
3	Baja	No hay verificación de email al cambiar el propio email.	Integrar el flujo estándar de verificación de Rock si el email cambia.
4	Baja	Validación del email en backend ausente — solo el front valida <code>required</code> , no <code>formato</code> .	Agregar regex mínima en <code>SaveMember</code> y rechazar con <code>ActionBadRequest</code> .

6.3 Observaciones de estilo

- Propiedades DTO en `camelCase` — no idiomático pero consistente; mantener por consumo del front.
- Mezcla de comentarios ES/EN — preferir ES para dominio, EN para helpers técnicos.
- `FamilyHub.obs` usa clases CSS con prefijos `vr*` y `fh*` sin scope `scoped`, lo que podría sangrar estilos. Todas las reglas son específicas por clase, riesgo bajo.
- El CSS sobrescribe `--color-interface-*` de Rock para forzar light-mode. Si Rock cambia esos tokens en una futura versión, revisar.

7. Seguridad / autorización

- **Autenticación:** cada `BlockAction` chequea `RequestContext.CurrentPerson` y rechaza con `ActionBadRequest("No autenticado.")`.
- **Autorización:** `CanEditPerson` permite editar solo a: 1. Miembros activos del grupo familiar primario del usuario. 2. Personas que sean miembros activos del Known Relationship group donde el usuario es Owner.
- **No hay edición de propio `Person.Email` con verificación** — si el usuario cambia su email, no se dispara flujo de verificación.
- **No hay auditoría explícita** — se confía en el history de Rock.

8. Puntos de entrada rápidos

Quiero...	Ir a
Cambiar el UI del modal	<code>FamilyHub.obs</code> <code><template></code> → <code>.fhmodal</code>
Agregar un campo al form	Ampliar <code>EditModelBag</code> (C#) + <code>EditModel</code> (TS) + <code>SaveMemberRequestBag</code> (C#)
Agregar un rol KR nuevo	Configurar <code>AvailableKnownRelationshipRoles</code> en el admin del bloque
Cambiar el default de código país	<code>editModel.value.phoneCountry = "502"</code> en <code>openNew()</code> y <code>SaveMobilePhoneV2</code> default
Cambiar el tamaño de thumbnail	<code>BuildPhotoUrl(photoId, 112)</code> en <code>GetPeopleMerged</code> ; edit modal usa 160
Cambiar la calidad JPEG	<code>SaveAsJpeg(..., 82L)</code> en <code>OptimizeProfileImageBinaryFile</code>

9. Tests manuales recomendados

1. **Usuario no autenticado** → ve NotificationBox "Debes iniciar sesión".
 2. **Usuario sin familia** → ve statusHtml de advertencia, sin grilla.
 3. **Agregar miembro nuevo (menor de edad)** → se inserta como Child en la familia.
 4. **Agregar miembro con rol KR de "Primo"** → se agrega al KR group, no a la familia.
 5. **Cambiar de KR "Primo" a Marital "Casado"** → tras el fix, ya no conserva el rol "Primo" residual.
 6. **Cambiar de Marital "Casado" a KR "Tío"** → limpia `MaritalStatusValueId` de ambos.
 7. **Editar a mí mismo** → el selector de parentesco no aparece; no se aplica marital.
 8. **Subir foto grande** → overlay "Subiendo foto..." ; tras 25s sin respuesta, libera el estado.
 9. **Guardar con teléfono vacío** → borra el `PhoneNumber` existente.
 10. **Guardar con email inválido** — hoy el backend no valida formato; el front solo valida required. Considerar agregar regex validator.
-

10. Dependencias Rock

- `Rock.Data.RockContext`
 - `Rock.Model.PersonService` , `GroupService` , `GroupMemberService` , `GroupTypeRoleService` , `PhoneNumberService` , `BinaryFileService`
 - `Rock.Web.Cache.GroupTypeCache` , `DefinedValueCache` , `BinaryFileTypeCache`
 - `Rock.SystemGuid.BinaryFiletype.PERSON_IMAGE`
 - `Rock.SystemGuid.DefinedValue.PERSON_PHONE_TYPE_MOBILE`
 - `Rock.SystemGuid.DefinedValue.PERSON_RECORD_TYPE_PERSON`
 - `Rock.SystemGuid.DefinedValue.PERSON_CONNECTION_STATUS_PARTICIPANT`
 - `Rock.SystemGuid.GroupRole.GROUPROLE_FAMILY_MEMBER_ADULT` / `CHILD`
 - `Rock.SystemGuid.GroupRole.GROUPROLE_KNOWN_RELATIONSHIPS_OWNER` / `CHILD`
-

11. Historial de cambios

Fecha	Cambio
2026-04-18	Fix: N+1 queries en <code>GetPeopleMerged</code> — ahora usa batching (≤ 5 queries totales en vez de $\sim 5+4N$).
2026-04-18	Fix: null-guards en todas las llamadas a <code>GroupTypeCache.GetFamilyGroupType()</code> vía el helper <code>GetFamilyGroupId()</code> .
2026-04-18	Rename: <code>DeactivateFamilyMembershipsExcept</code> → <code>RemoveFamilyMembershipsExcept</code> con docstring acorde al comportamiento real (borra).
2026-04-18	Fix: <code>EnsurePersonHasPrimaryFamily</code> ahora resuelve <code>adultRole</code> antes de crear el <code>Group</code> ; evita grupos huérfanos.
2026-04-18	Cleanup: eliminadas propiedades <code>mobile</code> de los bags y la función <code>parsePhone</code> del frontend (quedaban como compat obsoleta).
2026-04-18	Cleanup: eliminados helpers muertos <code>GetKnownRelationFromMe</code> y <code>GetMobilePhone</code> (reemplazados por batching).
2026-04-18	Refactor: <code>ApplyPhotoFromBag</code> unifica guardado de foto (se elimina código duplicado en Path A/B de <code>SaveMember</code>).
2026-04-18	Fix: al cambiar parentesco a estado civil, se limpian Known Relationships previos.
2026-04-18	Fix: al cambiar de estado civil a Known Relationship, se limpia <code>MaritalStatusValueId</code> en ambos adultos. La lógica anterior dependía de <code>GetMaritalRelationshipValue</code> que fallaba si el target ya no era Adulto activo o si la opción marital fue removida del bloque.
2026-04-18	Fix: <code>AddPersonToFamily</code> acepta <code>forceAdultRole</code> — en el contexto marital garantiza rol Adulto sin inferencia por edad. Evita que un cónyuge quede como Child si venía de KR con <code>BirthDate < 18</code> o con membresía Child previa.
2026-04-18	Documentación inicial (<code>FamilyHub.md</code>).